COMPTIA SECURITY+ SY0-701 | DOMAIN 2 · THREATS, VULNERABILITIES, AND MITIGATIONS (22%)

Security+ 2.4 · Password Attacks

Mestre CyberSec · **Lesson 2.4 — Password Attacks** (7:15) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Executive Lesson Summary: Password attack: password spraying (UMA senha comum MUITOS usuários — evita lockout); credential stuffing (senhas VAZADAS reusadas); brute force (todas as combinações — rápido mas detectável); dictionary (lista de palavras); rainbow table (hash pré-computados — MORTO pelo SALTING

💡 Key Exam Highlights & Critical Concepts

- Password spraying: 1 senha fraca contra muitos usuários (discreto)
- Brute force = tentar tudo; dictionary = lista de palavras
- Rainbow table = hash pré-computada — MORTA pelo SALTING
- Reusa de senha vazada em outros sites = credential stuffing

🎯 Exam Tips & Typical CompTIA Scenarios

- 'single weak password tried against many accounts' = password spraying
- 'pre-computed hash table' = rainbow table (defesa: salting)

🚨 Beyond the Video: Official Exam Concepts, Protocols & Traps (Section 2.4 — Password Attacks)

Why is this advanced coverage required for SY0-701?: Video lectures provide foundational concepts, but CompTIA tests practical engineering judgment and requires deep familiarity with the following protocols, acronyms, and operational nuances in this domain:

• Password Spraying (N/A)

An attack that tries a single common weak password against many different user accounts, often staying under lockout thresholds.

⚠️ ***Exam Trap:*** *Gatilho: 'uma senha fraca contra muitas contas' = password spraying. Não confunda com brute force/dictionary, que focam uma única conta.*

• Credential Stuffing (N/A)

Using breached username/password pairs from one service to authenticate to other services, exploiting password reuse.

⚠️ ***Exam Trap:*** *Gatilho: 'senhas vazadas reutilizadas em outros sites' = credential stuffing; difere de spraying porque usa pares usuário/senha vazados.*

• Brute Force Attack (N/A)

Trying every possible character combination until the correct password is found; can be online or offline.

⚠️ ***Exam Trap:*** *Gatilho: 'todas as combinações possíveis'. É barulhento/detectável online e pode causar lockout.*

• Dictionary Attack (N/A)

Using a list of common words, leaked passwords, and variations to guess passwords rather than trying all possible combinations.

⚠️ ***Exam Trap:*** *Gatilho: 'lista de palavras' ou 'wordlist'. Não tenta todas as combinações como o brute force.*

• Rainbow Table (N/A)

A precomputed table of password hashes used to quickly reverse unsalted password hashes.

⚠️ ***Exam Trap:*** *Gatilho: 'tabela de hashes pré-computados' = rainbow table; defesa principal: salting.*

• Salting (N/A)

Adding a unique random value to each password before hashing, stored with the hash, so identical passwords produce different hashes.

⚠️ ***Exam Trap:*** *Gatilho: salting torna rainbow tables inúteis. Não impede brute force, mas aumenta o custo; senhas iguais geram hashes diferentes.*

- **Password Hash (N/A)**

A one-way cryptographic representation of a password used for verification; if stolen, it can be attacked offline.

⚠️ ***Exam Trap:*** *Gatilho: hash não é criptografia; ataques offline usam hashes roubados. Salting e key stretching são defesas.*

- **Key Stretching (N/A)**

Applying repeated salted hashing to slow password verification and make brute-force or dictionary attacks more expensive.

⚠️ ***Exam Trap:*** *Gatilho: bcrypt, PBKDF2, Argon2 = key stretching. Defesa contra brute force/dicionário, especialmente offline.*

🏛️ Conceptual Deep Dive, Real-World Analogies & GRC Analysis

📌 Brute Force vs. Dictionary vs. Hybrid Attack

🏠 **Real-World Analogy:** Imagine trying to open a four-digit mechanical padlock. A Brute Force Attack is an exhaustive mathematician attempting every conceivable alphanumeric permutation sequentially: 0000, 0001, 0002... up to 9999; it is mathematically guaranteed to succeed eventually, but an 15-character complex string would take centuries. A Dictionary Attack is a pragmatic thief testing only real-world common words found in human language and common leak lists: 'admin', '123456', 'password', 'welcome'; it runs in milliseconds, but will fail against random strings like 'X#9kQ'. A Hybrid Attack merges both techniques: it ingests dictionary root words and systematically applies human substitution patterns (leet-speak substituting '@' for 'a' and appending trailing digits or exclamation marks, generating 'P@ssw0rd2024!').

🔍 **Root Cause & Background:** Cognitive limits cause humans to select predictable passphrases derived from everyday words and naive character substitutions. Modern distributed computing clusters and GPUs compute hundreds of billions of cryptographic hashes per second, cracking dictionary-based and predictable passwords in seconds.

🛡️ **GRC Auditor Perspective:** Align corporate identity policies with modern NIST SP 800-63B guidelines: mandate passphrase length (minimum 15-16 characters instead of counterproductive 30-day forced rotations); implement real-time screening against breached password blacklists; and enforce Multi-Factor Authentication (MFA) to nullify static credential compromises.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker attempts every possible alphanumeric combination sequentially to crack an encrypted password'. Answer: Brute Force Attack. Trigger: 'attacker ingests a pre-compiled wordlist of common phrases and terminology into an automated cracking tool'. Answer: Dictionary Attack. Trigger: 'attacker modifies wordlist entries by substituting characters with numbers and symbols (e.g., P@ssw0rd1!)'. Answer: Hybrid Attack.

📌 Password Spraying vs. Credential Stuffing

🏠 **Real-World Analogy:** Imagine a commercial complex with 1,000 locked office doors. If you stand before Suite 101 and attempt five incorrect keys in rapid succession, the security alarm trips and the lock disables for 30 minutes (account lockout policy). In Password Spraying, an intruder selects a single popular seasonal key (e.g., 'CompanySpring2024!') and tests it once on Door 1, once on Door 2, and once on Door 3 across all 1,000 offices. Because no single lock experiences multiple consecutive failures, no threshold alarms sound, and the intruder unlocks five vulnerable suites. In Credential Stuffing, the adversary acquires a dump of 50,000 email/password pairs breached from a retail store database and deploys automated bots to replay those identical credentials against high-value banking and enterprise portals, exploiting ubiquitous human password reuse.

🔍 **Root Cause & Background:** Password Spraying exploits traditional lockout thresholds configured on a per-account basis, evading detection by executing horizontal attempts (one guess across thousands of IDs). Credential Stuffing exploits human password reuse across disparate personal and corporate platforms combined with automated credential validation bots.

🛡️ **GRC Auditor Perspective:** Mitigating Password Spraying demands User and Entity Behavior Analytics (UEBA) flagging anomalous aggregate authentication surges using identical passwords across distinct accounts, paired with mandatory MFA. Countering Credential Stuffing requires automated compromised credential monitoring (integrating threat intelligence breach databases to force proactive resets on exposed accounts) and Web Application Firewalls (WAFs) capable of bot mitigation.

⚠️ **How CompTIA Tests This:** Trigger: 'attacker tests a single commonly used password across hundreds of user accounts horizontally to evade account lockout thresholds'. Answer: Password Spraying. Trigger: 'attacker deploys automated bots to test massive databases of leaked credentials harvested from prior third-party breaches against a target website'. Answer: Credential Stuffing.

📌 Rainbow Tables vs. Cryptographic Salting

🏠 **Real-World Analogy:** A Rainbow Table is a precomputed phone directory where cryptanalysts spent months mapping millions of plaintexts directly to their resulting hash values: 'if hash equals 5f4dcc3b, the underlying password was password'. An attacker holding a database dump does not compute hashes on the fly; they merely perform instantaneous reverse lookups. A Cryptographic Salt is like a chef tossing a unique, randomly generated secret seasoning into each individual dish before cooking: even if two patrons order identical base meals ('123456'), the random salt ensures the final biochemical signatures differ entirely. The attacker's precomputed lookup directory does not account for this unique per-user salt and becomes totally worthless.

🔍 **Root Cause & Background:** Standard cryptographic hash algorithms are deterministic: identical plaintexts always produce identical digest strings. In unsalted databases, adversaries leverage precomputed Rainbow Tables (which trade memory storage for compute cycles via reduction functions) to reverse millions of stolen password hashes in fractions of a second.

🛡️ **GRC Auditor Perspective:** Audit identity storage schemas to verify passwords are never hashed using unsalted or fast algorithms (such as raw MD5 or SHA-256). Mandate modern, compute-intensive Key Stretching algorithms (such as Argon2, bcrypt, or PBKDF2) paired with a cryptographically secure, unique pseudo-random Salt of at least 16 to 32 bytes generated per account.

⚠️ **How CompTIA Tests This:** Trigger: 'precomputed mathematical lookup table used to crack password hashes rapidly by trading storage space for compute time'. Answer: Rainbow Table. Trigger: 'cryptographic technique of appending unique random data to passwords before hashing to defeat rainbow tables and ensure identical passwords yield different digests'. Answer: Salt / Salting.

📌 Account Lockout Policies vs. Denial of Service (DoS) Vulnerability

🏠 **Real-World Analogy:** Imagine a bank branch establishing an unyielding rule: 'If any individual enters an incorrect door PIN three consecutive times, the branch immediately drops armored shutters and locks the entire building for the rest of the day.' A mischievous vandal strolls by at 7:00 AM, deliberately types three wrong PINs into every executive office door in 15 seconds, and walks away smiling. The rigid security policy designed to protect the business inadvertently paralyzes banking operations, creating a self-inflicted catastrophe.

🔍 **Root Cause & Background:** Rigid account lockout thresholds (e.g., locking an account after 3 consecutive failures) were originally engineered to thwart online brute force attempts. However, adversaries weaponize this mechanism: by scripting intentional failed login attempts against targeted administrative and executive accounts, they transform a defensive control into an operational Denial of Service (DoS) attack.

🛡️ **GRC Auditor Perspective:** In accordance with NIST SP 800-63B, GRC auditors should recommend deprecating aggressive hard account lockout thresholds in favor of resilient mitigations: progressive rate limiting (exponential throttling delays between failed attempts), CAPTCHA challenges, context-aware risk-based step-up authentication, and mandatory MFA.

⚠️ **How CompTIA Tests This:** Trigger: 'an aggressive account lockout setting allows an external adversary to intentionally disable legitimate employee accounts via scripted failed logins'. Answer: Denial of Service (DoS) risk. CompTIA / NIST mitigation: Progressive delays (throttling), CAPTCHA challenges, and risk-based authentication.

⚡ QUICK-REFERENCE EXAM CHEAT SHEET

Typical Exam Scenario / Practical Situation	Technical Concept & Target Answer
Exhaustive testing of every possible alphanumeric permutation sequentially until finding the correct password	**Brute Force Attack**
Using a pre-compiled wordlist of common terms and breached passwords fed into automated cracking tools	**Dictionary Attack**
Attacking thousands of distinct user accounts using a single popular password to stay under lockout thresholds	**Password Spraying**
Automated replay of massive databases of leaked email/password pairs from prior breaches against enterprise logins	**Credential Stuffing**
Precomputed reverse lookup table mapping hashes to plaintexts to crack passwords rapidly by trading memory for time	**Rainbow Table**
Appending unique cryptographically random strings to passwords before hashing to defeat rainbow table attacks	**Cryptographic Salting**

✅ Authentic Practice Questions (10 Real Exam Scenarios)

Scenario-based exam questions. Complete explanations and distractor analysis are provided in the answer key below.

QUESTION 1 · EXAM SCENARIO

A security analyst sees authentication logs where a single source IP attempted the password Summer2024! against 300 different user accounts. Each account received only one failed login, and no account was locked out. The attempts continued every 45 minutes with a different common password. Which password attack best matches this behavior?

A) Credential stuffing using credentials from a third-party breach

B) Password spraying to avoid account lockout thresholds

C) Offline brute-force attack against captured password hashes

D) Rainbow table lookup against unsalted password hashes

QUESTION 2 · EXAM SCENARIO

Employees at a company report unauthorized VPN logins from unfamiliar locations. Investigation shows the attacker used a list of email addresses and passwords taken from an unrelated social media breach. A few logins succeeded because employees had reused their personal passwords at work. Which attack occurred?

A) Dictionary attack

B) Password spraying

C) Credential stuffing

D) Brute-force attack

QUESTION 3 · EXAM SCENARIO

A penetration tester extracts password hashes from a legacy application that uses unsalted MD5. Instead of submitting login attempts, the tester uses a very large precomputed mapping of hash values to plaintext passwords to recover several passwords quickly. Which technique is being used?

A) Rainbow table attack

B) Password spraying

C) Key stretching

D) Credential stuffing

QUESTION 4 · EXAM SCENARIO

A developer is updating an application's password storage. The security team wants to ensure that two users who choose the same password do not produce the same stored hash, and that precomputed lookup tables cannot be reused against the database. Which change should the developer implement?

A) Enforce a 90-day password expiration policy

B) Require multifactor authentication for all users

C) Configure account lockout after three failed attempts

D) Add a unique random value to each password before hashing it

QUESTION 5 · EXAM SCENARIO

A company must store password hashes for a high-value internal application. The security architect wants the hashing process to remain slow and resource-intensive even on modern hardware, so that offline guessing of each password takes much longer. Which technique should be selected?

A) Key stretching with bcrypt or PBKDF2

B) Password spraying prevention

C) Credential stuffing detection

D) Rainbow table generation

QUESTION 6 · EXAM SCENARIO

An attacker captures a password hash database from a server and then uses a tool that systematically tries every possible combination of letters, numbers, and symbols until the correct plaintext is found for one administrator account. Which type of attack is this?

A) Dictionary attack

B) Credential stuffing

C) Brute-force attack

D) Password spraying

QUESTION 7 · EXAM SCENARIO

During a web application assessment, a tester uses a list containing common passwords, seasons, and variations of the company name to guess user passwords. The tester tries the list against a single known username until a match is found. Which attack is being performed?

A) Rainbow table attack

B) Dictionary attack

C) Brute-force attack

D) Password spraying

QUESTION 8 · EXAM SCENARIO

A financial services firm allows users to authenticate to an internal portal. Attackers repeatedly try one common password across many employee accounts, waiting between attempts so that each account stays below the lockout threshold. Which control most directly mitigates this specific attack without relying only on account lockout?

A) Multifactor authentication

B) Increasing password maximum length

C) Disabling password history

D) Storing passwords with reversible encryption

QUESTION 9 · EXAM SCENARIO

An organization discovers that several user accounts were compromised because employees used the same passwords on corporate systems and personal websites that were later breached. The attackers simply replayed the leaked username and password pairs. Which additional control would best prevent this from succeeding in the future?

A) Password complexity requirements

B) Shorter password expiration intervals

C) Multifactor authentication

D) Unique salts in the password hash database

QUESTION 10 · EXAM SCENARIO

A security auditor finds that a database stores SHA-256 password hashes without a salt. The auditor warns that attackers could recover many passwords at once if the database is stolen. Which password attack is most directly mitigated by adding a unique salt to each password before hashing?

A) Password spraying

B) Credential stuffing

C) Dictionary attack

D) Rainbow table attack

ANSWER KEY & COMPREHENSIVE DISTRACTOR EXPLANATIONS

Question 1: Option B — Password spraying to avoid account lockout thresholds

Correct: Password spraying tries a few common passwords across many accounts to stay under lockout thresholds. The scenario shows one attempt per account across 300 accounts with repeated common passwords, not breached credential pairs, per-account brute force, or offline hash lookup. Credential stuffing would use previously breached username/password pairs; brute force would make many attempts per account; rainbow tables are offline precomputed hash attacks.

Question 2: Option C — Credential stuffing

Correct: Credential stuffing uses username/password pairs stolen from other breaches and replays them against a target, relying on password reuse. The scenario explicitly involves an unrelated breach and successful logins due to reused passwords. Password spraying uses common passwords across many accounts; dictionary uses wordlists; brute force enumerates all possibilities.

Question 3: Option A — Rainbow table attack

Correct: A rainbow table is a precomputed mapping of password hashes to plaintext, effective against unsalted hashes. Salting or key stretching with unique salts defeats precomputed tables. Password spraying and credential stuffing are online credential attacks, not offline precomputed hash lookups.

Question 4: Option D — Add a unique random value to each password before hashing it

Correct: Salting adds unique random data to each password before hashing, so identical passwords produce different hashes and precomputed tables cannot be reused. Password expiration, MFA, and lockout are useful controls but do not change hash uniqueness or defeat rainbow table reuse against stolen hashes.

Question 5: Option A — Key stretching with bcrypt or PBKDF2

Correct: Key stretching, such as bcrypt, PBKDF2, or Argon2, intentionally slows hashing with iterations or work factors, making offline guessing expensive. Password spraying and credential stuffing are attack methods, not defenses. Rainbow table generation is an attack; salting alone prevents precomputation but key stretching adds computational cost.

Question 6: Option C — Brute-force attack

Correct: Brute force systematically tries all possible combinations, which can eventually find a short or weak password but is noisy and slow. Dictionary uses a focused wordlist; credential stuffing replays breached pairs; password spraying tries a few common passwords across many accounts to avoid lockout.

Question 7: Option B — Dictionary attack

Correct: A dictionary attack uses a curated list of likely passwords and variations, often faster than exhaustive brute force against human-chosen passwords. Rainbow tables require captured hashes; brute force tries all combinations; password spraying targets many accounts with a few passwords.

Question 8: Option A — Multifactor authentication

Correct: MFA most directly mitigates password spraying because even if the common password is guessed for an account, the attacker still needs a second factor. Password length and complexity can help but do not directly stop the attack pattern; password history and reversible encryption are poor controls. Account lockout alone can be bypassed by spraying below thresholds across many accounts.

Question 9: Option C — Multifactor authentication

Correct: MFA prevents credential stuffing from succeeding with only a leaked username and password, because the attacker cannot provide the second factor. Complexity and expiration may reduce weak passwords but do not stop valid breached credentials from being replayed; salting protects stored hashes, not online authentication with valid leaked credentials.

Question 10: Option D — Rainbow table attack

Correct: Rainbow tables rely on precomputed hash values. A unique salt per password changes the hash output, so precomputed tables become ineffective. Password spraying and credential stuffing are online attacks using guessed or leaked credentials; dictionary attacks can still attempt guesses, but salting specifically defeats precomputed lookup tables.